Agent Workforce Control Plane (AWCP)

45-Day Implementation Blueprint

Domain Focus: Data Science & Intelligence (Team Size: 3)

Objective: Build the Cognitive Shield, Memory Substrate, and Recovery Reasoning Engine

1 Week 1: State Normalization & The Memory Substrate

The first week establishes how the control plane perceives, standardizes, and hashes the chaotic output from various autonomous agents. The 3 Data Scientists (DS-1, DS-2, DS-3) will work in parallel to build the foundational data structures.

1.1 Task Allocation & Implementation Steps

- **DS-1: Workflow State Assembly Logic.** Your objective is to build the ingestion intelligence that parses raw runtime logs. You must define the **Governing Slice**—the absolute minimum combination of metadata, owner identity, and active feature flags required to judge if a workflow is behaving safely, deliberately leaving out irrelevant conversation history to save processing power. You will map out **Declared Write Scopes**, which act as pre-approved digital boundaries detailing exactly which external systems (like a billing API or a CRM) an agent is allowed to alter.
- **DS-2: Context Graph Manager Prototyping.** You will begin designing the **Context Graph**. Instead of storing an agent's memory as a flat text transcript, you will model it as a Directed Acyclic Graph (DAG) where nodes are facts, API responses, or user intents. This prevents the system from losing track of important details over long tasks. You must build relevance scoring algorithms to determine which nodes are most critical to the current step.
- **DS-3: Cryptographic Context Hashing.** You will develop the **Context Hash** function. Every time an agent makes a move, you must take a snapshot of its current memory and state, passing it through a hashing algorithm (like SHA-256) to create a unique string of characters. This ensures that if the agent makes a mistake and we need to rewind, we have a mathematically verifiable, tamper-proof record of exactly what the agent "knew" at that exact millisecond.

Weekly Study & Literature Focus

Topics to master: Vector embeddings for structured logs; Graph databases (e.g., Neo4j or Letta memory frameworks) for managing multi-agent shared memory; Semantic hashing techniques where minor whitespace changes in logs do not drastically alter the state hash.

2 Week 2: Recursive Compression & Trace Summarization

With raw data normalized, the team must now solve the "context window problem." Autonomous agents generate massive trace files that will quickly overwhelm standard LLM limits if not compressed intelligently.

2.1 Task Allocation & Implementation Steps

- **DS-1: Recursive Language Models (RLMs) Integration.** You will configure the **LLM Gateway** (an interface unifying Anthropic and OpenAI models) to perform **Context Folding**. This is a recursive compression technique where a 100-page trace is summarized into 10 pages, and those 10 pages are summarized into 1 page, ensuring the agent retains the overarching goal and critical decision points without drowning in minor details.
- **DS-2: Artifact Ledger Folding.** Agents often execute code that returns massive JSON payloads. You will build extraction logic that parses these payloads, extracts only the state-changing variables, and writes them into the **Evidence Ledger**—the durable, append-only database used for audits and replays.

- **DS-3: Stale-Context Detection Engine.** You will build the intelligence that detects hallucinations caused by outdated memory. By comparing the agent's current working memory against the chronologically verified Evidence Ledger, your model will flag **Stale Context**—situations where an agent is trying to act on information (like a customer's refund eligibility) that has already been altered by a previous step.

Weekly Study & Literature Focus

Topics to master: Chain-of-Density (CoD) prompting to generate highly concentrated summaries; Map-Reduce pipelines for LLM context compression; Temporal logic for identifying chronological contradictions in text generation.

3 Week 3: Failure Budgeting & Autonomy Thresholds

This week, the data science team transitions from memory management to safety mechanics, designing the mathematical models that dictate when an agent must be stopped.

3.1 Task Allocation & Implementation Steps

- **DS-1: Failure Budget Modeling.** You will define the mathematical formulas for the **Failure Budget Breach**. You need to calculate a dynamic threshold based on API timeouts, 400/500 HTTP error rates, and logic loops. When an agent exceeds this budget, it triggers the control plane's safety mechanisms.
- **DS-2: Progressive Autonomy Reduction Ladder.** You will design the state transition matrix for autonomy. When the failure budget is breached, the agent is forced into **Graceful Degradation**. You will define the rules for moving an agent into **Conservative Mode** (where it can only perform read-only analysis and suggest low-risk routes) and **Recommendation-Only Mode** (where all tool-writing capabilities are disabled until a human clicks approve).
- **DS-3: Trace Sampling Multipliers.** Working with the DevOps metrics, you will write the logic that dynamically increases **Trace Sampling** depth. In standard operation, we might log 10% of the agent's thought process to save money. Your model will automatically spike this to 100% capture depth the moment a failure signal is detected, ensuring we have full visibility right before a crash.

Weekly Study & Literature Focus

Topics to master: Control theory (PID controllers) applied to software rate-limiting; Open Policy Agent (OPA) and Rego language syntax for mapping your mathematical thresholds to hard-coded backend rules; Risk-tier classification algorithms.

4 Week 4: The Recovery & Replay Intelligence

When an agent fails, the system must allow a human operator to rewind the state, fix the issue, and resume. The DS team must build the reasoning layers that make this time-travel possible.

4.1 Task Allocation & Implementation Steps

- **DS-1: Replay Synthesis Logic.** You will write the prompts and evaluation pipelines that consume data from the Evidence Ledger to generate a human-readable **Replay Summary**. This summary must clearly state what the agent was trying to do, which API failed, and what the **Next Safe Action** should be, presenting this cleanly to the UI team.
- **DS-2: Safe Resume Checkpointing.** You will design the algorithm that scans the Context Graph to find the last known **Safe Resume Point**. This is a specific contextual snapshot where the agent had perfectly verified data before it made the catastrophic error, allowing the Python backend to restart the task from that exact millisecond without repeating previous, successful API calls.
- **DS-3: Cross-Agent Context Transfer.** You will build the intelligence for the **Handoff Coordinator**. When a failed task is diverted from a primary agent to a specialized remediation agent, your models must ensure **Session Continuity**—meaning the new agent perfectly inherits the exact "Governing Slice" of memory it needs to fix the problem, without context loss.

Weekly Study & Literature Focus

Topics to master: Deterministic state machine recovery; Multi-agent communication protocols; Information bottlenecks in neural networks (understanding what data is strictly necessary for task continuation).

5 Week 5: Automated Instrumentation & Code Reasoning

The Control Plane is responsible for managing "wild" agents that other teams build. If an agent is uploaded without safety tracking, the DS team uses code-generation models to automatically fix it.

5.1 Task Allocation & Implementation Steps

- **DS-1: Codex Fine-Tuning for Telemetry Patches.** You will utilize models like OpenAI Codex or Claude to build the **Patch Generator**. When an agent is flagged as under-instrumented, your pipeline will read the agent's Python code and generate a Pull Request (PR) that injects missing **Observability Hooks**—digital markers that force the agent to emit a signal every time it starts or finishes a step.
- **DS-2: Quarantine Exit Validation.** You will build an LLM-as-a-Judge evaluation framework for the **Quarantine Policy**. This model will analyze the generated telemetry patches to mathematically verify if the new code successfully covers all write-capable functions before the agent is allowed to execute tasks in production.
- **DS-3: Narrow Token Risk Evaluation.** You will refine the prompt architecture for the **Expiring Approval Tokens**. When an agent asks to make a massive database change, your model must evaluate the "blast radius" of the request, assign it a risk score, and generate a narrow, time-boxed window (e.g., "Approved to write only to the billing table for the next 5 minutes").

Weekly Study & Literature Focus

Topics to master: Abstract Syntax Tree (AST) manipulation and parsing in Python; Few-shot prompting for code-generation models (LLMs writing code); LLM-as-a-Judge frameworks for automated code review.

6 Week 6: System Orchestration, Dry Runs & Tuning

In the final week before the midway milestone, all data science models must be unified, optimized for cost/latency, and proven to work alongside the Python, DevOps, and UI components.

6.1 Task Allocation & Implementation Steps

- **DS-1: Provider Pinning & Gateway Load Balancing.** Working inside the **LLM Gateway**, you will implement dynamic model routing. For simple log summarization, you will route requests to cheaper, faster models. However, when a critical failure occurs, you will enforce **Provider Pinning**, forcefully routing the recovery logic to the most capable reasoning model (e.g., Claude 3.5 Sonnet) regardless of cost, to guarantee high-fidelity recovery.
- **DS-2: Safer Profile Overrides.** You will write the logic that integrates the Data Science models with the DevOps feature-flags. When the system detects a failure, your algorithms will automatically toggle the flags that shift the agent into a **Safer Profile**—for example, forcing the agent to use a highly-restrictive set of tools or limiting its API concurrency to 1 request per second.
- **DS-3: 90-Day Proof Milestone Evaluation.** You will design the final testing sweep. You must simulate a massive cross-system write failure, proving that the control plane correctly gates the action, gracefully reduces the agent's autonomy, isolates the branch, and presents a perfectly hashed, replayable summary to the UI without dropping the workflow.

Weekly Study & Literature Focus

Topics to master: API rate limiting and token economics (cost tracking across multiple LLM providers); Chaos engineering principles applied to autonomous agents; End-to-end latency optimization for synchronous LLM calls.

Agent Workforce Control Plane (AWCP)

45-Day Implementation Blueprint

Domain Focus: Python Backend & Orchestration (Team Size: 2)

Objective: Build the Durable Orchestration, Security Sandboxing, and Governance Integration

7 Week 1: Orchestration Foundations & The Intake Proxy

The backend team must build the "front door" of the control plane and the foundational engine that keeps workflows alive during crashes. The 2 Python Engineers (PB-1, PB-2) will split ingestion and orchestration tasks.

7.1 Task Allocation & Implementation Steps

- **PB-1: Workflow Intake Proxy Construction.** You will build the entry point using FastAPI or gRPC. Your core task is building Runtime Adapters, which act as translators that convert chaotic, runtime-specific signals (like an AutoGPT log vs. a LangChain event) into a single, standardized **Workflow Identity**. This ensures the control plane speaks one universal language regardless of which agent initiated the action.
- **PB-2: Temporal Cluster Initialization.** You will configure and deploy **Temporal**, our durable workflow execution engine. Temporal fundamentally changes how Python scripts run by automatically saving the state of local variables to a database after every step. This means if the server physically catches fire in the middle of a process, the workflow will simply resume exactly where it left off on a new server without repeating steps.
- **PB-1 & PB-2: Event Ingestion Pipeline.** Together, you will connect the Intake Proxy to a message broker like Kafka or NATS. This decouples the agent's speed from our processing speed, utilizing **Load Shaping**—a technique that smooths out massive spikes in agent activity by holding incoming requests in a queue, preventing our downstream LLM APIs from hitting rate limits.

Weekly Study & Literature Focus

Topics to master: Temporal SDK for Python (specifically understanding Workers, Activities, and Workflows); API Gateway design patterns; Event-driven architecture using Kafka/NATS for decoupled microservices.

8 Week 2: Durable Execution & State Machines

With the foundation set, the team must structure how agentic tasks are actually executed safely across multiple steps and external systems.

8.1 Task Allocation & Implementation Steps

- **PB-1: DAG Modeling & Safe Resume Points.** You will model all agent workflows as Directed Acyclic Graphs (DAGs)—mathematical structures ensuring tasks only flow forward without getting stuck in infinite loops. You will program **Safe Resume Points**, which force the Temporal engine to write a permanent checkpoint to the database every time the Data Science team's model verifies an agent's context is safe.
- **PB-2: Saga Semantics & Compensating Transactions.** You will implement the **Saga Pattern**. Since agents often do multi-step cross-system writes (e.g., creating a user in the CRM *and* updating the billing API), if step two fails, we cannot leave the system in a half-finished state. You will write "compensating transactions"—automated rollback scripts that undo step one if step two fails, ensuring data consistency.
- **PB-1 & PB-2: Idempotency Key Management.** You will build a middleware that attaches **Idempotency Keys** to every single agent tool call. This is a unique, one-time identifier ensuring that if a network timeout causes an agent to accidentally send a "refund customer $50" command twice, the external API recognizes the duplicate key and only processes the refund once.

Weekly Study & Literature Focus

Topics to master: The Saga design pattern for distributed transactions; Idempotency in REST APIs; Graph traversal algorithms in Python.

9 Week 3: Sandboxed Execution & CodeAct

Agents do not just make API calls; they often write and execute raw Python or TypeScript to solve problems. The backend team must build a secure cage for this behavior.

9.1 Task Allocation & Implementation Steps

- **PB-1: Modal Ephemeral Sandboxing.** You will integrate **Modal**, a serverless compute platform, to build the **CodeAct Sandbox**. When an agent writes code, it is highly dangerous to run it on our main servers. You will configure Modal to spin up an **Ephemeral Container**—a temporary, highly restricted virtual machine that runs the agent's code, returns the output, and immediately destroys itself to prevent any malicious lateral movement.
- **PB-2: Trace & Artifact Capture.** Inside the sandbox, you will intercept everything the agent does. You will build the **Artifact Ledger Fold** logic. When the sandboxed code returns a massive API payload or diff patch, your Python script will compress it and write it directly into the Evidence Ledger, ensuring the UI team and Data Science models have access to the exact outputs of the execution.

Weekly Study & Literature Focus

Topics to master: Serverless container orchestration (Modal, AWS Firecracker); Secure sandboxing and isolating execution environments (namespaces, cgroups); Inter-process communication (IPC).

10 Week 4: Policy Enforcement & Token Gates

The Python team must now integrate the Data Science team's math and the infrastructure's safety rules into hard-coded enforcement gates.

10.1 Task Allocation & Implementation Steps

- **PB-1: Open Policy Agent (OPA) Integration.** You will connect the workflow engine to OPA using the **Approval Gate Controller**. OPA uses a language called Rego to evaluate policies. You will write the Python wrappers that pause an agent's workflow, query OPA with the agent's current state, and completely block the execution path if OPA returns a policy violation (e.g., "Agent exceeded its write-scope limit").
- **PB-2: Expiring Approval Tokens.** For high-risk actions, you will implement a **Durable Wait**. Instead of failing instantly, the workflow pauses and issues a request for a **Narrow Approval Token**—a time-boxed cryptographic ticket (e.g., valid for 10 minutes) generated by the system once a human operator clicks "Approve" in the UI. Your Python code will resume the workflow only if this exact token is presented.

Weekly Study & Literature Focus

Topics to master: Open Policy Agent (OPA) and Rego language syntax; JWT (JSON Web Tokens) for time-boxed, stateless authorization; Asynchronous polling and Webhook patterns in FastAPI.

11 Week 5: Agent Handoffs & Context Middleware

When an agent fails, it often hands the task over to a specialized "remediation agent." The backend must ensure this transfer is seamless.

11.1 Task Allocation & Implementation Steps

- **PB-1: Handoff Coordinator Implementation.** You will integrate the **agent-session-bridge** middleware. Your task is to build the routing logic that ensures **Session Continuity**. When Agent A hands off to Agent B, your backend must pull the exact Context Hash from the Data Science models and inject it directly into Agent B's initialization payload, guaranteeing zero context loss during the transition.
- **PB-2: Branch Isolation Logic.** You will implement **Branch Isolation**. If a workflow has parallel tasks (e.g., analyzing data while simultaneously querying an API), and one task crashes, your Temporal workflows must be designed to isolate the failed thread. This prevents the crash from corrupting the shared memory graph, allowing the healthy branches to continue working while the failed branch waits for recovery.

Weekly Study & Literature Focus

Topics to master: Concurrency and parallel execution in Temporal; Middleware design patterns in Python; Multi-agent framework architectures (e.g., LangGraph or AutoGen underlying mechanics).

12 Week 6: Integration, Telemetry & Final Proofing

In the final week before the midway milestone, the backend must be wired perfectly to the UI, Data Science outputs, and DevOps telemetry tools.

12.1 Task Allocation & Implementation Steps

- **PB-1: OpenTelemetry (OTel) Instrumentation.** You will weave **OpenTelemetry hooks** throughout the entire Python codebase. This means every time a function starts, makes a database call, or evaluates a policy, it emits a standardized "Trace Span." This gives the UI team the exact data they need to visualize the workflow's health and gives the DevOps team the metrics needed to monitor the platform.
- **PB-2: Feature Flag Wire-up.** You will integrate a feature flag provider (like OpenFeature or Flipt). Your code will dynamically check these flags before execution. If the Data Science model detects a failure and flips a flag, your Python backend will instantly switch the agent to a **Safer Profile**, physically blocking access to certain tools or severely throttling the agent's API request limits.
- **PB-1 & PB-2: 90-Day Proof Milestone Execution.** You will co-lead the system-wide dry run. You must ingest a simulated workflow, route a state-changing action through your OPA gate, pause the Temporal workflow durably until an approval token is manually generated, and successfully execute a sandboxed write in Modal without losing the context hash.

Weekly Study & Literature Focus

Topics to master: OpenTelemetry (Traces, Metrics, and Context Propagation in Python); Feature Flagging architectural patterns; End-to-end integration testing in distributed systems.

Agent Workforce Control Plane (AWCP)

45-Day Implementation Blueprint

Domain Focus: UI & Operator Surface (Team Size: 1)

Objective: Build the Risk-First Dashboard, Evidence Viewer, and Intervention Workbenches

13 Week 1: The Risk-First Control Center Foundations

Unlike traditional task managers that show everything an agent is doing, the AWCP UI must be strictly "Risk-First." The solo UI Engineer (UI-1) will establish the core layout that forces human operators to focus only on broken, blocked, or risky agent behaviors.

13.1 Task Allocation & Implementation Steps

- **UI-1: Live Control Surface Initialization.** You will construct the main dashboard using React or Vue. Your primary task is to build the high-level telemetry widgets that poll the Python Backend every 15 seconds. You must create dedicated, color-coded zones for **Workflows in Degraded Mode** (agents that have breached failure budgets) and **Pending Branch Tokens** (agents paused waiting for human permission to write data).
- **UI-1: State & Telemetry Polling.** Instead of using heavy WebSockets for everything, you will implement a smart polling mechanism that queries the Temporal workflow engine's state. You will build the **Drift Sweep** trigger—a manual button that operators can click to instantly sweep all active agents and check if their current actions are drifting away from their **Declared Write Scopes**.

Weekly Study & Literature Focus

Topics to master: State management in React (Zustand/Redux) for rapidly changing polling data; UX patterns for incident response dashboards (e.g., PagerDuty/Datadog design systems); Asynchronous UI updates without blocking the main thread.

14 Week 2: The Recovery Workbench & Evidence Viewer

When an agent crashes, the operator needs to see exactly why. The UI must present the massive amounts of data generated by the Data Science team in a clean, human-readable format.

14.1 Task Allocation & Implementation Steps

- **UI-1: Replay Summary Visualization.** You will build the **Recovery Workbench**. When an operator clicks on a degraded workflow, you must render the **Replay Summary** generated by the DS models. This means creating a chronological timeline component that maps **Trace Spans** (the exact millisecond-by-millisecond logs of what the agent did) to specific policy decisions.
- **UI-1: Context Hash & JSON Inspector.** You will integrate a code/JSON inspector tool into the workbench. Operators need to inspect the **Context Hash** (the unique fingerprint of the agent's memory) and the **Artifact Ledger Fold** (the compressed API outputs). This requires building a collapsible, searchable tree-view component that can handle large, deeply nested JSON objects without lagging the browser.

Weekly Study & Literature Focus

Topics to master: Virtualized lists for rendering massive JSON/log files in the DOM (e.g., react-window); Chronological timeline UI architectures; Data visualization for tracing metrics.

15 Week 3: The Approval Gate & Token Interfaces

The UI must provide the specific screens where human operators physically approve or deny high-risk actions that the Python backend has paused.

15.1 Task Allocation & Implementation Steps

- **UI-1: Expiring Token Approval Queue.** You will build the interface for the **Approval Gate**. When an agent tries to execute a cross-system write (like deleting an account), the UI must display a high-alert card. This card must clearly show the proposed tool plan, the blast radius, and a countdown timer for the **Narrow Approval Token** (e.g., "This request expires in 14:59").
- **UI-1: Next-Safe Action Prompts.** Working with the Data Science team's outputs, you will build dynamic action buttons. If an agent is in **Recommendation-Only Mode**, the UI shouldn't just show an error; it must render the **Next Safe Action** calculated by the LLM (e.g., "Queue Manual Review" or "Resume from Checkpoint t+08:14").

Weekly Study & Literature Focus

Topics to master: Optimistic UI updates (updating the UI before the server confirms the token generation to make the app feel instantly responsive); Designing high-stakes confirmation modals (preventing accidental clicks).

16 Week 4: Quarantine Exit & Patch Review Flow

When an under-instrumented agent is blocked by the infrastructure, the UI must allow developers to review the automated code fixes generated by the control plane.

16.1 Task Allocation & Implementation Steps

- **UI-1: Quarantine Inventory View.** You will create a specific catalog view for **Visible But Write-Blocked Agents**. These are agents that have been registered but lack necessary telemetry hooks. The UI must clearly indicate their status as "Quarantined" with a strict read-only badge.
- **UI-1: Instrumentation Patch Diff-Viewer.** You will integrate a code-diff visualization library (similar to GitHub's PR viewer). When the DS team's Codex model generates an **Instrumentation Patch** (adding missing OpenTelemetry hooks to an agent's Python code), the operator must be able to read the proposed code changes side-by-side, leave comments, and click "Merge & Promote."

Weekly Study & Literature Focus

Topics to master: Implementing code diffing algorithms in the browser (e.g., using diff-match-patch or Monaco Editor integration); Building interactive code-review interfaces.

17 Week 5: Incident Channel Hooks & Operator UX

The Control Plane is not an isolated tool; it must integrate directly into the workflows of the DevOps and Agent Ops teams where they already communicate.

17.1 Task Allocation & Implementation Steps

- **UI-1: Incident Channel Deep-Linking.** You will build the "Open Incident Channel" functionality. When a workflow severely breaches its failure budget, the UI must allow the operator to instantly create a Slack or MS Teams channel. Crucially, you will build deep-links so that clicking an alert in Slack opens the AWCP dashboard exactly to the **Safe Resume Point** and corresponding **Evidence Ledger** entry.
- **UI-1: Autonomy Floor Visualizers.** You will create graphical indicators (like gauges or progress bars) that represent the **Autonomy Floor**—the minimum allowable independence an agent has before the system forces a hard stop (e.g., displaying "Autonomy: 20% - Forced Degradation Active").

Weekly Study & Literature Focus

Topics to master: OAuth and API integrations for Slack/Teams; Deep linking and routing architectures in single-page applications (SPAs); Accessible data visualization (SVG/Canvas gauges).

18 Week 6: End-to-End Integration & Final Proofing

In the final week before the 90-day proof halfway point, the UI must smoothly connect all the pipes built by the Python and DS teams.

18.1 Task Allocation & Implementation Steps

- **UI-1: API Gateway Wire-up.** You will replace all mocked data with live endpoints from the Python backend's FastAPI. You must handle all loading states, network timeouts, and Temporal workflow waiting states gracefully, ensuring the operator never sees a blank screen if an agent's sandboxed code takes 30 seconds to execute.
- **UI-1: UX Flow Testing for the 90-Day Proof.** You will conduct the end-to-end dry run from the operator's perspective. You will manually simulate discovering a degraded workflow on the Live Control Surface, opening the Replay Summary, reviewing the Context Hash, evaluating a Narrow Approval Token for a database write, and clicking approve to resume the Temporal workflow safely.

Weekly Study & Literature Focus

Topics to master: Error boundary implementation in React/Vue; Fallback UI design for long-polling timeouts; End-to-end frontend testing (e.g., Cypress or Playwright) for complex multi-step workflows.

Agent Workforce Control Plane (AWCP)

45-Day Implementation Blueprint

Domain Focus: DevOps & Infrastructure (Team Size: 1)

Objective: Build the Control Substrate, Evidence Boundary, and Security Sandboxing

19 Week 1: The Control Substrate & Evidence Boundary

The solo DevOps Engineer (DO-1) is responsible for provisioning the underlying infrastructure that makes the AWCP durable and tamper-proof. The first priority is establishing the foundational storage and execution engines.

19.1 Task Allocation & Implementation Steps

- **DO-1: Provisioning the Temporal Cluster.** You will deploy a production-ready **Temporal** cluster on Kubernetes. Temporal is the durable workflow engine that ensures agent tasks survive network failures and server crashes. You must configure the persistence layer (typically Cassandra or PostgreSQL) to handle the massive state-saving requirements of thousands of concurrent autonomous agents.
- **DO-1: Establishing the Evidence Ledger.** You will set up the **Evidence Boundary**—a secure, append-only perimeter where all agent inputs, tool calls, and outputs are captured. You will provision an Object Store (like AWS S3 or MinIO) to serve as the **Replayable Evidence Ledger**. This ledger must be strictly immutable; once an agent's Context Hash or action trace is written here, it cannot be altered, ensuring perfect auditability during incident recovery.

Weekly Study & Literature Focus

Topics to master: Temporal architecture (History Service, Matching Service, FrontEnd Service); Kubernetes StatefulSets and persistent volume claims (PVCs) for database durability; Immutable storage architectures and WORM (Write Once, Read Many) policies in cloud object stores.

20 Week 2: Security Sandboxing & The CodeAct Environment

Agents frequently generate and execute raw Python or TypeScript to interact with APIs. Running this code directly on control plane servers is a massive security risk. You must build an isolated cage for these actions.

20.1 Task Allocation & Implementation Steps

- **DO-1: Modal Integration for Ephemeral Compute.** You will integrate **Modal** (or a similar serverless container platform) to handle the **CodeAct Sandbox**. When an agent writes code to execute a state-changing tool call, you will configure the infrastructure to spin up an **Ephemeral Container**. This is a highly restricted, temporary virtual machine that executes the code, returns the output back to the Evidence Ledger, and immediately self-destructs, preventing any malicious lateral movement across the network.
- **DO-1: Network Egress & Branch Isolation Controls.** You will configure strict network policies for these containers. By implementing **Branch Isolation**, you ensure that if one agent's workflow goes rogue and attempts to exfiltrate data, the failure is contained to that specific ephemeral sandbox, preventing the crash or security breach from corrupting the shared memory graph of other active agents.

Weekly Study & Literature Focus

Topics to master: Firecracker microVMs and serverless container orchestration; Kubernetes Network Policies (Calico/Cilium) for strict egress control; Principle of Least Privilege (PoLP) in automated IAM roles.

21 Week 3: Observability & The Telemetry Backbone

The control plane cannot govern what it cannot see. You must deploy the infrastructure that captures every thought, API call, and error generated by the agent workforce.

21.1 Task Allocation & Implementation Steps

- **DO-1: OpenTelemetry (OTel) Collector Deployment.** You will deploy a fleet of **OpenTelemetry Collectors**. These collectors act as giant vacuums, sucking up the "Trace Spans" (millisecond-level logs of an agent's execution path) emitted by the Python backend. You must configure the routing logic to send high-level metrics to the UI dashboard and raw, heavy traces directly into the Evidence Ledger.
- **DO-1: Dynamic Trace Sampling Depth.** Storing 100% of agent logs is prohibitively expensive. You will configure dynamic **Trace Sampling Depth** rules. Under normal conditions, the infrastructure might only retain 10% of the logs. However, you will wire this to the Data Science team's failure models: the exact millisecond an agent breaches its failure budget, your infrastructure will automatically spike the capture rate to 100x (full depth), ensuring operators have perfectly detailed logs leading up to the crash.

Weekly Study & Literature Focus

Topics to master: OpenTelemetry architecture (Receivers, Processors, Exporters); Head-based vs. Tail-based sampling strategies for distributed tracing; PromQL/Metrics querying for real-time dashboarding.

22 Week 4: Feature Flags & Governance Policies

To physically stop an agent from making a mistake, the infrastructure needs a mechanism to instantly toggle the agent's permissions in real-time.

22.1 Task Allocation & Implementation Steps

- **DO-1: OpenFeature / Flipt Deployment.** You will deploy a dedicated feature-flag control plane (like OpenFeature or Flipt). This allows the system to change an agent's behavior without deploying new code. You will configure the flags that control **Safer Profiles**—pre-defined, heavily restricted execution environments. If an agent starts failing, the system toggles a flag that instantly revokes its access to dangerous APIs.
- **DO-1: Open Policy Agent (OPA) Server Setup.** You will deploy the **Open Policy Agent (OPA)** server, which evaluates the Data Science team's mathematical rules. OPA acts as the ultimate authority for the **Approval Gate**. You will ensure OPA operates with ultra-low latency so that when the Python backend asks, "Is this agent allowed to execute this cross-system write?", OPA instantly returns a cryptographic yes or no based on the active feature flags and declared write scopes.

Weekly Study & Literature Focus

Topics to master: Feature toggle architecture in highly concurrent systems; Open Policy Agent (OPA) deployment and caching strategies; Rego policy testing and CI/CD integration.

23 Week 5: The Quarantine & Patch Pipeline

Agents built by external teams will often connect to the control plane missing crucial safety features. The DevOps engineer must build the automated system that blocks them and demands fixes.

23.1 Task Allocation & Implementation Steps

- **DO-1: Quarantine Entry Routing.** You will build the **Quarantine Policy** enforcement at the API Gateway layer. When the Intake Proxy detects a new agent (e.g., "claims-triage-v1") that lacks required OpenTelemetry hooks or OPA callbacks, your ingress controllers will instantly route it into a "Quarantined" state. This physically blocks the agent from executing any governed write-capable work.

- **DO-1: CI/CD Pipeline for Instrumentation Patches.** You will connect the Data Science team's Codex-generated patches to your GitOps pipeline (e.g., GitHub Actions or ArgoCD). When the LLM generates an **Instrumentation Patch** (a proposed code fix to add missing safety hooks), your CI/CD pipeline will automatically open a Pull Request against the offending agent's repository and run automated tests to ensure the new hooks actually emit data before allowing the agent to exit quarantine.

Weekly Study & Literature Focus

Topics to master: GitOps workflows (ArgoCD, Flux); Automating Pull Requests via CI/CD pipelines; Kubernetes Ingress routing and API Gateway rate-limiting based on headers/metadata.

24 Week 6: End-to-End Orchestration & Load Testing

In the final week before the midway milestone, the infrastructure must be stress-tested to ensure it can handle the chaotic, unpredictable load of autonomous agent workflows without buckling.

24.1 Task Allocation & Implementation Steps

- **DO-1: LLM Gateway Failover & Provider Pinning.** You will configure the **OpenClaw** (or equivalent LLM Gateway) routing rules. During a system crash, you must guarantee high-fidelity recovery. You will configure **Provider Pinning**, a routing rule that forcefully directs all incident recovery prompts to the most reliable model (e.g., Anthropic Claude), bypassing cheaper, lower-tier models, and configure automatic failovers if an LLM provider experiences an outage.
- **DO-1: Chaos Engineering for the 90-Day Proof.** You will execute chaos engineering tests (using tools like Gremlin or Chaos Mesh) alongside the Python and DS teams. You will intentionally kill Temporal worker nodes, simulate API timeouts, and trigger **Failure Budget Breaches** to mathematically prove that the system automatically degrades autonomy, preserves the Replayable Evidence Ledger, and safely issues Narrow Approval Tokens without dropping the workflow state.

Weekly Study & Literature Focus

Topics to master: Chaos Engineering principles in Kubernetes; LLM API Gateway configurations (retry logic, circuit breakers, timeout thresholds); Load testing distributed systems (Locust, k6).

Agent Workforce Control Plane (AWCP)

45-Day Implementation Blueprint

Domain Focus: Cross-Domain Orchestration & Business Governance

Objective: Unified System Integration, Scenario Testing, and The 45-Day Midway Proof

25 Weeks 1 & 2: The Governance Baseline

The first two weeks require the entire team to establish the "Proof Slice"—the exact boundaries of what the 90-day trial will cover. Without strict cross-team alignment, the individual domains will build disjointed systems.

25.1 Task Allocation & Implementation Steps

- **Target Workflow Selection:** All leads (DS, Python, UI, DevOps) must agree on the single existing agent workflow to govern first (e.g., "refund-branch-882"). Attempting to govern multiple workflows simultaneously in the first 45 days will cause scope creep.
- **Stack Discipline Enforcement:** The team must establish the strict boundary between the "Core" (identity, policy, evidence ledger) and the "Replaceable" (the specific LLM, the feature flag tool). This ensures the architecture does not become hard-coded to a single vendor.
- **Shared Vocabulary Sync:** The Data Science concept of a "Governing Slice," the Python concept of a "Temporal Checkpoint," and the UI concept of a "Replay Summary" must be mapped to exact JSON schemas so all four domains are communicating over the exact same data structures.

Cross-Domain Study & Literature Focus

Topics to master: Domain-Driven Design (DDD) for unified terminology (Ubiquitous Language); API Contract Testing to ensure DS models and Python backends don't break each other's integrations.

26 Weeks 3 & 4: The Simulation & Integration Phase

As individual components come online, the team must begin "stitching" the pipes together, focusing heavily on how failure moves through the system.

26.1 Task Allocation & Implementation Steps

- **Data-to-Backend Handoff:** The Python team must successfully pass a live runtime event through the Temporal engine, hand it to the Data Science relevance scorer, and receive a mathematically verified "Safe" or "Unsafe" signal back within acceptable latency limits (e.g., under 800ms).
- **Backend-to-Infra Sandboxing:** The DevOps engineer and Python team must prove that a Python script generated by the LLM can be securely handed off to the Modal Ephemeral Container, executed, and the results folded back into the Evidence Ledger without breaking the Temporal loop.
- **Infra-to-UI Telemetry Loop:** The UI engineer must connect the live OpenTelemetry spans and OpenFeature flag states to the dashboard, ensuring that when the DevOps engineer manually flips a "Safer Profile" flag in the backend, the UI dashboard instantly turns red and updates the Autonomy Floor visualizer.

Cross-Domain Study & Literature Focus

Topics to master: Distributed Tracing across polyglot microservices; System Integration Testing (SIT); Latency budgeting in LLM-augmented architectures.

27 Weeks 5 & 6: The 45-Day Midway Proof (The Dry Run)

To ensure the team is on track for the ultimate 90-day business proof, the 45-day mark requires a full-stack, cross-domain demonstration of the three core Agent Ops Scenarios.

27.1 Task Allocation & Implementation Steps

- **Simulating Scenario A (Graceful Degradation):** The team forces an agent to fail three write attempts. The system must automatically increase trace sampling (DevOps), trigger a safer profile (DS/Backend), and display the degraded workflow on the Risk-First Dashboard (UI) without killing the workflow entirely.
- **Simulating Scenario B (Approval Tokens):** An agent proposes a massive database change. The OPA policy engine must block it (Backend), evaluate the blast radius (DS), and issue a narrow approval request. The operator must review the context diff and manually click approve (UI), allowing the workflow to resume and execute securely (DevOps).
- **Simulating Scenario C (Onboarding & Quarantine):** A "rogue" agent without telemetry hooks is connected. The ingress controllers must quarantine it immediately (DevOps). The Codex model must generate an instrumentation patch (DS), and the operator must review the proposed code diff and merge it (UI) before the agent is allowed to execute.

Cross-Domain Study & Literature Focus

Topics to master: Incident Response war-gaming; Demonstrating technical ROI to non-technical stakeholders (VP of Agent Ops); Formulating the final 45-day sprint backlog based on dry-run failures.